

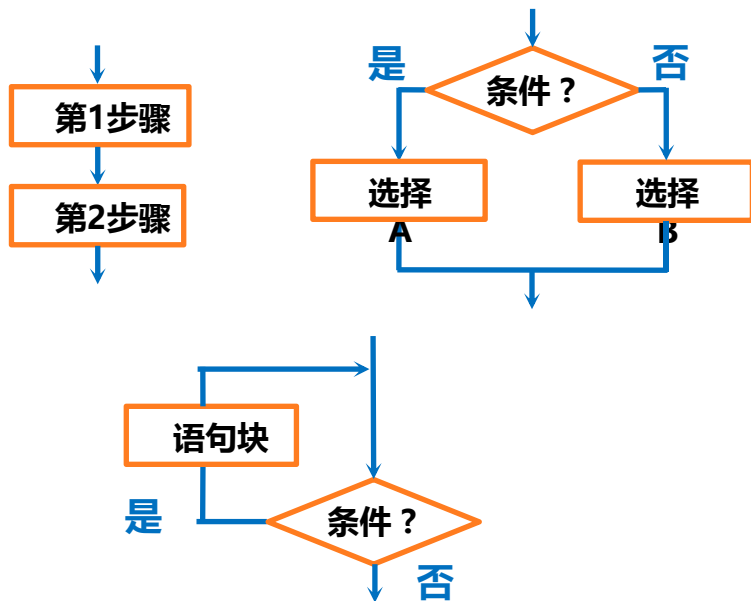
第4章 程序的控制结构





本章概要

章节内容



- 4.1 分支结构
- 4.2 异常结构
- 4.3 实例5: 身体质量指数BMI
- 4.4 循环结构
- 4.5 条件计算
- 4.6 实例6: 圆周率的计算
- 4.7 random库和随机数运用

学习目标



- (1) 掌握程序的分支结构**
- (2) 掌握异常结构并处理程序异常**
- (3) 掌握程序的循环结构**
- (4) 运用循环控制解决复杂循环计算问题**
- (5) 掌握判断条件的构造方法**
- (6) 理解蒙特卡罗计算方法**
- (7) 掌握随机库的使用方法以及随机数的运用方法**



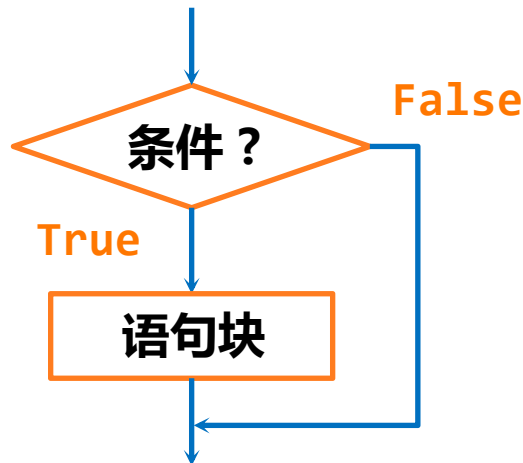
4.1 分支结构



单分支结构

根据判断条件结果而选择不同向前路径的运行方式

if <条件> :
 <语句块>



单分支结构



单分支示例

```
guess = eval(input())
```

```
if guess == 99:  
    print("猜对了")
```

```
if True:  
    print("条件正确")
```



单分支练习

练习1: 某商家促销，购物满200打9折，输入购物金额， 如果购物金额超过200元，就计算并显示折扣后的金额（打9折）

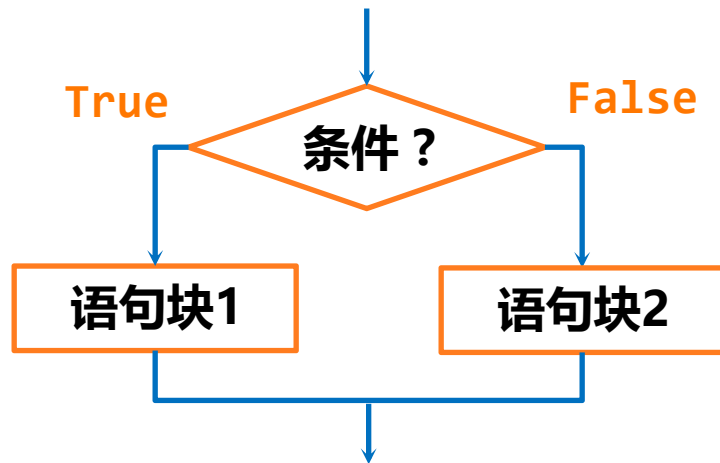
练习2: 输入学生成绩，如果成绩大于等于60分，就打印"恭喜你，及格了！"



二分支结构

根据判断条件结果而选择不同向前路径的运行方式

```
if <条件> :  
    <语句块1>  
else :  
    <语句块2>
```



二分支结构



二分支示例

```
guess = eval(input())
```

```
if guess == 99:
```

```
    print("猜对了")
```

```
else :
```

```
    print("猜错了")
```

```
if True:
```

```
    print("语句块1")
```

```
else :
```

```
    print("语句块2")
```

二分支结构



紧凑形式：适用于简单表达式的二分支结构

〈表达式1〉 *if* 〈条件〉 *else* 〈表达式2〉

```
guess = eval(input())
```

```
print("猜{}了".format("对" if guess==99 else "错"))
```

二分支结构



二分支练习

练习1：输入一个成绩，判断成绩是否及格

```
score = eval(input())
if score >= 60:
    print("恭喜你及格了")
else :
    print("很遗憾，不及格")
```

练习2：输入一个数字，判断它是奇数还是偶数

```
num = eval(input())
if num%2==0:
    print(num,"是偶数")
else :
    print(num,"是奇数")
```



二分支结构

二分支练习

练习3：回文数判断。设 n 是一个任意自然数，如果 n 的各位数字反向排列:所得自然数与 n 相等，则 n 被称为回文数。从键盘输入一个 5 位数字，请编写程序判断这个数字是不是回文数。如：12321

```
num = input("请输入一个 5 位数字：")
if num == num[::-1]:
    print("{} 是回文数".format(num))
else:
    print("{} 不是回文数".format(num))
```



多分支结构

if <条件1> :

<语句块1>

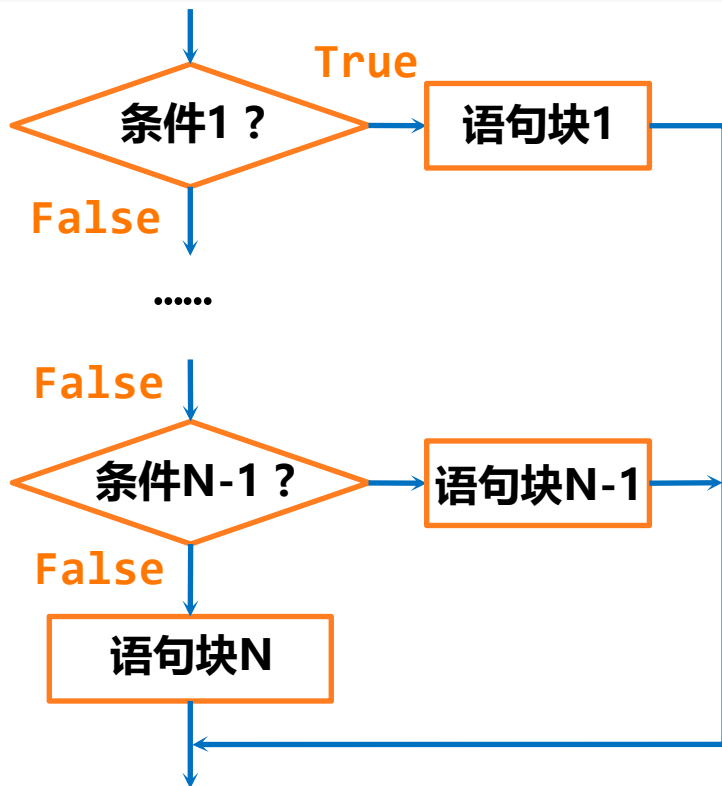
elif <条件2> :

<语句块2>

.....

else :

<语句块N>





多分支结构

对不同分数分级的问题

```
score = eval(input())
if score >= 60:
    grade = "D"
elif score >= 70:
    grade = "C"
elif score >= 80:
    grade = "B"
elif score >= 90:
    grade = "A"
print("输入成绩属于级别{}".format(grade))
```

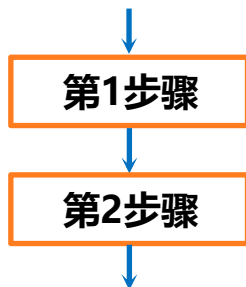
- 注意多条件之间的包含关系

- 注意变量取值范围的覆盖

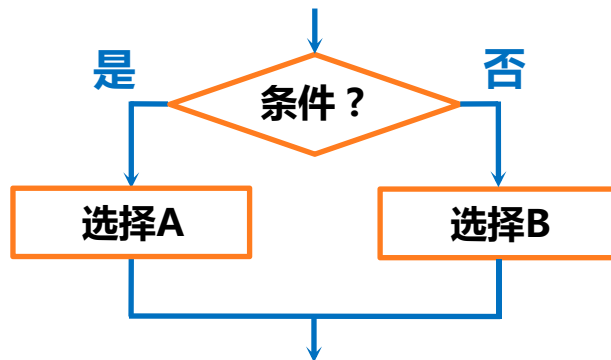


"程序的控制结构"

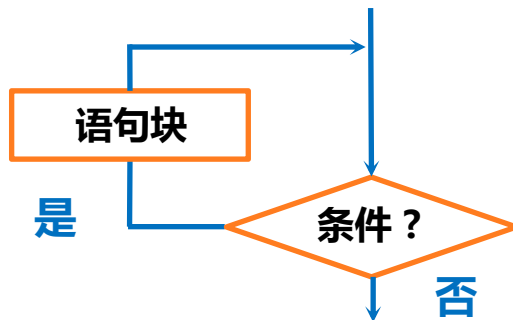
- 顺序结构



- 分支结构



- 循环结构





单元小结：分支结构

- 单分支 *if*
- 二分支 *if-else* 及紧凑形式
- 多分支 *if-elif-else* 及条件之间关系

单元小结：分支结构注意事项

1. 冒号和缩进
2. 布尔值的正确表达
3. 条件顺序很重要
4. 比较运算符错误（比较两个对象是否相等用 “==” ）
5. 合理使用括号分组

```
if (age >= 20 and sex="female") or (age >=22 and sex="male"):
    print("合法结婚")
```

4.2 异常结构

异常处理



```
num = eval(input("请输入一个整数: "))  
print(num**2)
```

当用户没有输入整数时，会产生异常，怎么处理？

异常发生的代码行数

Traceback (most recent call last):

File "t.py", line 1, in <module>

num = eval(input("请输入一个整数: "))

File "<string>", line 1, in <module>

NameError: name 'abc' is not defined

异常类型

异常内容提示

异常处理的基本使用

try :

＜语句块1＞

except :

＜语句块2＞

try :

＜语句块1＞

except ＜异常类型＞ :

＜语句块2＞

示例1

try :

```
num = eval(input("请输入一个整数: "))
```

```
print(num**2)
```

except :

```
print("输入不是整数")
```

示例2

try :

```
num = eval(input("请输入一个整数: "))
```

```
print(num**2)
```

except NameError:

```
print("输入不是整数")
```

标注异常类型后，仅响应此类异常
异常类型名字等同于变量名



异常处理

示例3

try:

可能引发异常的代码

```
num = int(input("请输入一个数字: "))
```

```
result = 100 / num
```

```
print(f"结果是: {result}")
```

except ValueError:

```
print("错误: 请输入有效的数字! ")
```

except ZeroDivisionError:

```
print("错误: 不能除以零! ")
```

try :

＜语句块1＞

except :

＜语句块2＞

else :

＜语句块3＞

finally :

＜语句块4＞

异常处理的高级使用

- *else* 对应语句块3在不发生异常时执行
- *finally* 对应语句块4一定执行



try:

```
num = int(input("请输入一个数字: "))  
result = 100 / num
```

except ValueError:

```
print("错误: 请输入有效的数字! ")
```

except ZeroDivisionError:

```
print("错误: 不能除以零! ")
```

else:

```
print("计算成功, 结果是: ", result) # 如果没有异常发生, 则执行else块
```

finally:

```
print("无论是否发生异常, 都会执行finally块。") # 通常用于释  
放资源, 如关闭文件
```



单元小结：异常处理

- 异常和错误： `TypeError`, `NameError`
- 异常处理 *`try-except`*
- 异常处理 *`else-finally`*

4.3 实例5: 身体质量指数BMI

身体质量指数BMI



BMI：对身体质量的刻画

- BMI: Body Mass Index

国际上常用的衡量人体肥胖和健康程度的重要标准，主要用于统计分析

- 定义

$$\text{BMI} = \text{体重 (kg)} / \text{身高}^2 (\text{m}^2)$$

身体质量指数BMI



BMI：对身体质量的刻画

- 实例：体重 72 kg 身高 1.75 m

BMI 值是 23.5

- 这个值是否健康呢？



身体质量指数BMI

国际：世界卫生组织 国内：国家卫生健康委员会

分类	国际BMI值 (kg/m ²)	国内BMI值 (kg/m ²)
偏瘦	<18.5	<18.5
正常	18.5 ~ 24.9	18.5 ~ 23.9
偏胖	25 ~ 29.9	24 ~ 27.9
肥胖	≥30	≥28

身体质量指数BMI



问题需求

- 输入：给定体重和身高值
- 输出：BMI指标分类信息(国际和国内)



思路方法

- 难点在于同时输出国际和国内对应的分类
- 思路1：分别计算并给出国际和国内BMI分类
- 思路2：混合计算并给出国际和国内BMI分类



身体质量指标BMI

```
#CalBMIv1.py
```

```
height, weight = eval(input("请输入身高(米)和体重(公斤)[逗号隔开]: "))
```

```
bmi = weight / pow(height, 2)
```

```
print("BMI 数值为: {:.2f}".format(bmi))
```

```
who = ""
```

```
if bmi < 18.5:
```

```
    who = "偏瘦"
```

```
elif bmi <= 24.9:
```

```
    who = "正常"
```

```
elif bmi <= 29.9:
```

```
    who = "偏胖"
```

```
else:
```

```
    who = "肥胖"
```

```
print("BMI 指标为:国际'{0}'".format(who))
```

分类	国际BMI值	国内BMI值
偏瘦	<18.5	<18.5
正常	18.5 ~ 24.9	18.5 ~ 23.9
偏胖	25 ~ 29.9	24 ~ 27.9
肥胖	≥30	≥28



身体质量指标BMI

```
#CalBMIv2.py
```

```
height, weight = eval(input("请输入身高(米)和体重(公斤)[逗号隔开]: "))
```

```
bmi = weight / pow(height, 2)
```

```
print("BMI 数值为: {:.2f}".format(bmi))
```

```
nat = ""
```

```
if bmi < 18.5:
```

```
    nat = "偏瘦"
```

```
elif bmi <= 23.9:
```

```
    nat = "正常"
```

```
elif bmi <= 27.9:
```

```
    nat = "偏胖"
```

```
else:
```

```
    nat = "肥胖"
```

```
print("BMI 指标为:国内'{0}'".format(nat))
```

分类	国际BMI值	国内BMI值
偏瘦	<18.5	<18.5
正常	18.5 ~ 24.9	18.5 ~ 23.9
偏胖	25 ~ 29.9	24 ~ 27.9
肥胖	≥30	≥28

#CalBMIV3.py

```
height, weight = eval(input("请输入身高(米)和体重(公斤)[逗号隔开]: "))
```

```
bmi = weight / pow(height, 2)
```

```
print("BMI 数值为: {:.2f}".format(bmi))
```

```
who, nat = "", ""
```

```
if bmi < 18.5:
```

```
    who, nat = "偏瘦", "偏瘦"
```

```
elif 18.5 <= bmi <= 23.9:
```

```
    who, nat = "正常", "正常"
```

```
elif 24 <= bmi <= 24.9:
```

```
    who, nat = "正常", "偏胖"
```

```
elif 25 <= bmi <= 27.9:
```

```
    who, nat = "偏胖", "偏胖"
```

```
elif 28 <= bmi < 29.9:
```

```
    who, nat = "偏胖", "肥胖"
```

```
else:
```

```
    who, nat = "肥胖", "肥胖"
```

```
print("BMI 指标为:国际'{0}', 国内'{1}'".format(who, nat))
```

分类	国际BMI值	国内BMI值
偏瘦	<18.5	<18.5
正常	18.5 ~ 24.9	18.5 ~ 23.9
偏胖	25 ~ 29.9	24 ~ 27.9
肥胖	≥30	≥28



关注多分支条件的组合

- 多分支条件之间的覆盖是重要问题
- 程序可运行，但如果不正确，要注意多分支
- 分支结构是程序的重要框架，读程序先看分支

4.4 循环结构



遍历某个结构形成的循环运行方式

```
for <循环变量> in <遍历结构> :  
    <语句块>
```

- 从遍历结构中逐一提取元素，放在循环变量中

遍历循环



for <循环变量> *in* <遍历结构> :
 <语句块>

- 由保留字for和in组成，完整遍历所有元素后结束
- 每次循环，所获得元素放入循环变量，并执行一次语句块



计数循环(N次)

```
for i in range(N) :
```

<语句块>

- 遍历由range()函数产生的数字序列，产生循环

遍历循环的应用



计数循环(N次)

```
>>> for i in range(5):  
    print(i)
```

0

1

2

3

4

```
>>> for i in range(5):  
    print("Hello:",i)
```

Hello: 0

Hello: 1

Hello: 2

Hello: 3

Hello: 4



计数循环(特定次)

```
for i in range(M,N,K) :
```

<语句块>

- 遍历由range()函数产生的数字序列，产生循环

遍历循环的应用



计数循环(特定次)

```
>>> for i in range(1,6):  
    print(i)
```

1
2
3
4
5

```
>>> for i in range(1,6,2):  
    print("Hello:",i)
```

Hello: 1
Hello: 3
Hello: 5



遍历循环的应用

字符串遍历循环

```
for c in s :
```

＜语句块＞

- s是字符串，遍历字符串每个字符，产生循环

遍历循环的应用



字符串遍历循环

```
>>> for c in "Python123":  
    print(c, end=",")
```

P,y,t,h,o,n,1,2,3,

思考:

如果把语句块换成`print(c.upper(), end=",")`呢? P,Y,T,H,O,N,1,2,3,

如果把语句块换成`print(ord(c), end=",")`呢? 83,124,119,107,114,113,52,53,54,



列表遍历循环

```
for item in ls :
```

<语句块>

- ls是一个列表，遍历其每个元素，产生循环



列表遍历循环

```
>>> for item in [123, "PY", 456] :  
        print(item, end=",")
```

123,PY,456,



文件遍历循环

```
for line in fi :
```

<语句块>

- fi是一个文件标识符，遍历其每行，产生循环

遍历循环的应用



文件遍历循环

```
>>> for line in fi :  
        print(line)
```

优美胜于丑陋
明了胜于隐晦
简洁胜于复杂

优美胜于丑陋
明了胜于隐晦
简洁胜于复杂

遍历循环



for <循环变量> *in* <遍历结构> :
 <语句块>

- 计数循环(N次)
- 列表遍历循环
- 计数循环(特定次)
- 文件遍历循环
- 字符串遍历循环
-



由条件控制的循环运行方式

while <条件> :



<语句块>

- 反复执行语句块，直到条件不满足时结束

无限循环的应用



无限循环的条件

```
>>> a = 3
>>> while a > 0 :
    a = a - 1
    print(a)
```

2

1

0

```
>>> a = 3
>>> while a > 0 :
    a = a + 1
    print(a)
```

4

5

... (CTRL + C 退出执行)



break 和 continue

- **break**跳出并结束当前整个循环，执行循环后的语句
- **continue**结束当次循环，继续执行后续次数循环
- **break**和**continue**可以与**for**和**while**循环搭配使用

循环控制保留字



break 和 continue

```
>>> for c in "PYTHON" :  
    if c == "T" :  
        continue  
    print(c, end="")
```

PYHON

```
>>> for c in "PYTHON" :  
    if c == "T" :  
        break  
    print(c, end="")
```

PY



循环控制保留字

```
>>> s = "PYTHON"
>>> while s != "" :
    for c in s :
        print(c, end="")
    s = s[:-1]
```

PYTHONPYTHOPYTHPYTPYP

```
>>> s = "PYTHON"
>>> while s != "" :
    for c in s :
        if c == "T" :
            break
        print(c, end="")
    s = s[:-1]
```

PYPYPYPYP

- *break* 仅跳出当前最内层循环



循环与else

for <变量> *in* <遍历结构> :
 <语句块1>

else :
 <语句块2>

while <条件> :
 <语句块1>

else :
 <语句块2>



循环与else

- 当循环没有被break语句退出时，执行else语句块
- else语句块作为"正常"完成循环的奖励
- 这里else的用法与异常处理中else用法相似



循环与else

```
>>> for c in "PYTHON" :  
    if c == "T" :  
        continue  
    print(c, end="")  
else:  
    print("正常退出")
```

PYTHON正常退出

```
>>> for c in "PYTHON" :  
    if c == "T" :  
        break  
    print(c, end="")  
else:  
    print("正常退出")
```

PY



单元小结：循环结构

- *for...in* 遍历循环: 计数、字符串、列表、文件...
- *while* 无限循环
- *continue* 和 *break* 保留字: 退出当前循环层次
- 循环 *else* 的高级用法: 与 *break* 有关

4.5 条件计算

用于条件组合的三个保留字

操作符及使用	描述
x and y	两个条件x和y的逻辑 与
x or y	两个条件x和y的逻辑 或
not x	条件x的逻辑 非



逻辑运算

运算符	含义	举例	结果
and (与)	如果a为假, a and b 就返回假, 否则返回b的值	a=3 b=6 a and b	6
or (或)	如果a非0, 返回a的值, 否则返回b的值	a or b	3
not (非)	如果a 为真, 就返回假	not a	False

对于**and**, 如果没有假值, 返回的是最后一个真值, 如果有假值, 则返回的是第一个假值。

对于**or**, 如果没有真值, 返回的是最后一个假值, 如果有真值, 则返回的是第一个真值

```
>>> a=3
>>> b=6
>>> a and b
6
>>> a or b
3
>>> not a
False
>>>
```

运算示例

逻辑运算	运算结果
True and False	False
0 or False	False
not 5	False
True and False or True	True



比较运算

操作符

操作符	数学符号	描述
<	<	小于
<=	≤	小于等于
>=	≥	大于等于
>	>	大于
==	=	等于
!=	≠	不等于



条件判断及组合

示例

```
guess = eval(input())
if guess > 99 or guess < 99:
    print("猜错了")
else :
    print("猜对了")
```



条件判断及组合

示例

判断整数x能否被3整除且个位数字为5 `x%3==0 and x%10==0`

有某个年份year, 判断是否为闰年

`(year%4==0 and year%100!=0) or year%400==0`

扩展思考:

请输出1-100之间能被3整除且个位数字为5的数字

请输出2000-2050之间的闰年



单元小结：条件计算

- 逻辑运算 *not and or*
- 比较运算 *> >= == <= < !=*

4.6 实例6: 圆周率的计算



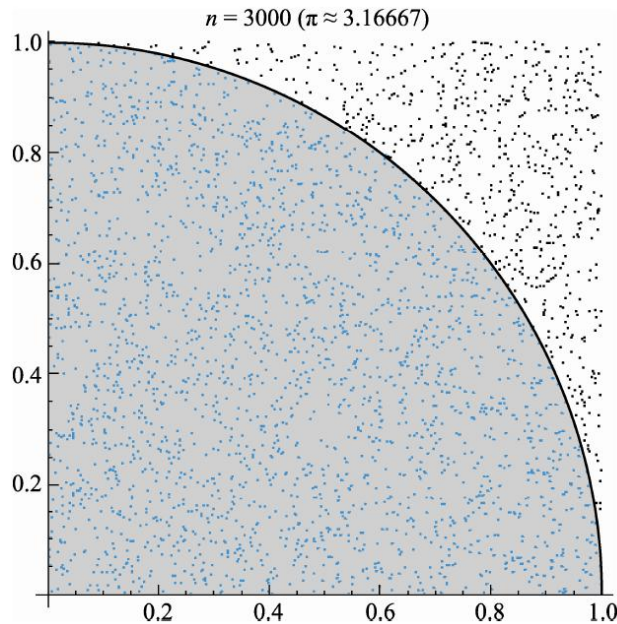
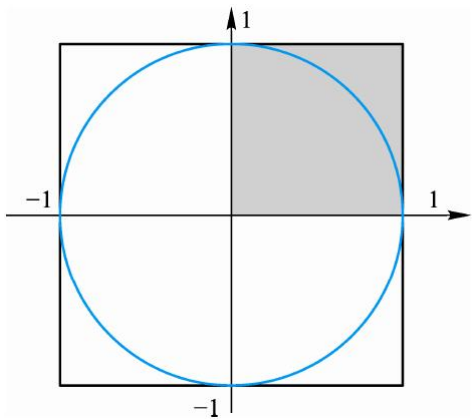
"圆周率的计算"问题分析

圆周率的近似计算公式

$$\pi = \sum_{k=0}^{\infty} \left[\frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \right]$$

"圆周率的计算"问题分析

蒙特卡罗方法





"圆周率的计算"实例讲解

圆周率的近似计算公式

$$\pi = \sum_{k=0}^{\infty} \left[\frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \right]$$

#CalPiV1.py

pi = 0

N = 100

for k **in** range(N) :

pi += 1/**pow**(16,k)*(\
4/(8*k+1) - 2/(8*k+4) - \
1/(8*k+5) - 1/(8*k+6))

print("圆周率值是: {}".format(pi))

$$\pi = \sum_{k=0}^{\infty} \left[\frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \right]$$

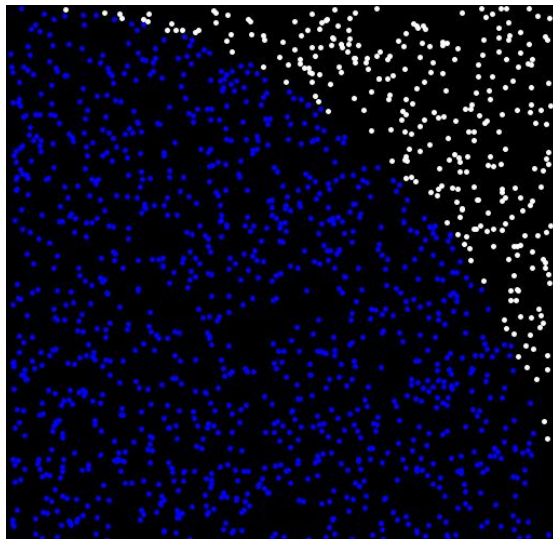
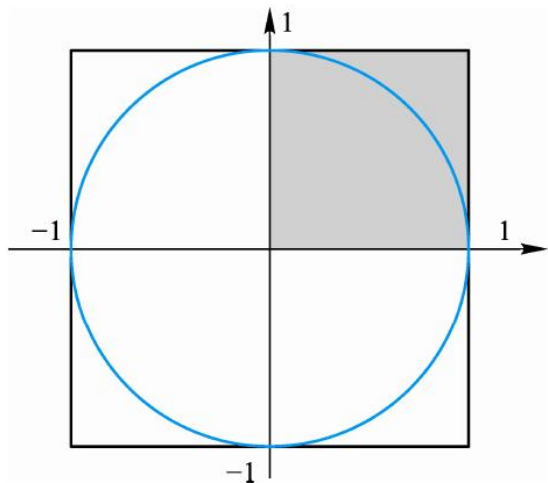
圆周率值是: 3.141592653589793



"圆周率的计算"实例讲解

蒙特卡罗方法

蒙特卡洛方法是一种基于随机抽样的数值计算方法，用于求解具有随机性的问题或近似计算确定性问题的数值解。



```
#CalPiV2.py
```

```
from random import random
```

```
from time import perf_counter
```

```
DARTS = 1000*1000
```

```
hits = 0.0
```

```
start = perf_counter()
```

```
for i in range(1, DARTS+1):
```

```
    x, y = random(), random()
```

```
    dist = pow(x ** 2 + y ** 2, 0.5)
```

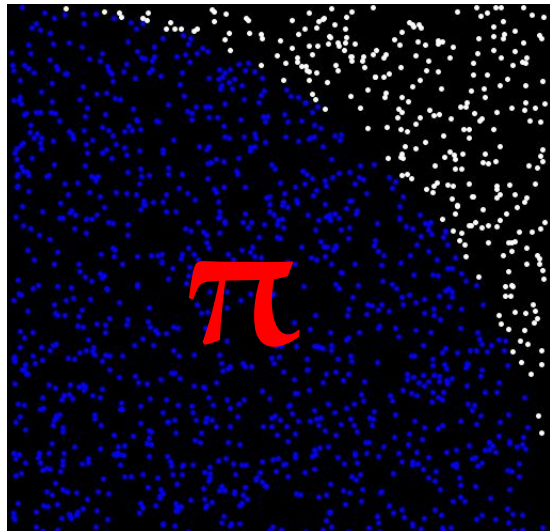
```
    if dist <= 1.0:
```

```
        hits = hits + 1
```

```
pi = 4 * (hits/DARTS)
```

```
print("圆周率值是: {}".format(pi))
```

```
print("运行时间是: {:.5f}s".format(perf_counter()-start))
```





理解方法思维

- 数学思维：找到公式，利用公式求解
- 计算思维：抽象一种过程，用计算机自动化求解
- 谁更准确？（不好说...）



"圆周率的计算"问题分析

圆周率的近似计算公式:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$



"圆周率的计算"问题解答

```
#CalPi-Leibniz-for
t=0
N=10000
pi=0
for i in range(N):
    pi+=(-1)**i/(2*i+1)
pi*=4
print(f"圆周率为: {pi}")
```

```
#CalPi-Leibniz-while
t=1.0
sign=1
pi=0
while abs(1/t)>1e-5:
    pi+=sign*1/t
    t+=2
    sign=-sign
pi*=4
print(f"圆周率为: {pi}")
```

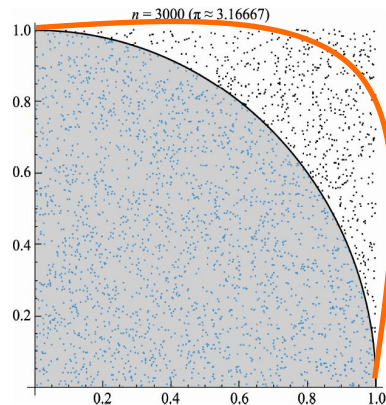


程序运行时间分析

- 使用time库的计时方法获得程序运行时间
- 改变撒点数量，理解程序运行时间的分布
- 初步掌握简单的程序性能分析方法

计算问题的扩展

- 不求解圆周率，而是某个特定图形的面积
- 在工程计算中寻找蒙特卡罗方法的应用场景



4.7 random库和随机数运用



random库概述

random库是使用随机数的Python标准库

- 伪随机数: 采用梅森旋转算法生成的(伪)随机序列中元素
- random库主要用于生成随机数
- 使用random库: `import random`



random库概述

random库包括两类函数，常用共8个

- **基本随机数函数：** `seed()`, `random()`
- **扩展随机数函数：** `randint()`, `getrandbits()`, `uniform()`,
`randrange()`, `choice()`, `shuffle()`

random库中的常用函数

函数	描述
<code>seed(a=None)</code>	初始化随机数种子，调用后会重新开始一个随机序列
<code>random()</code>	生成一个[0.0, 1.0)之间的随机小数
<code>randint(a, b)</code>	生成一个[a,b]之间的整数
<code>getrandbits(k)</code>	生成一个k比特长度的随机整数
<code>randrange(start, stop[, step])</code>	生成一个[start, stop)之间以step为步数的随机整数
<code>uniform(a, b)</code>	生成一个[a, b]之间的随机小数
<code>choice(seq)</code>	从序列类型(例如：列表)中随机返回一个元素
<code>shuffle(seq)</code>	将序列类型中元素随机排列，返回打乱后的序列



基本随机数函数

随机数种子

随机数种子

10

梅森旋转算法

随机
序
列

0.5714025946899135

0.4288890546751146

0.5780913011344704

0.20609823213950174

0.81332125135732 随机数

0.8235888725334455

0.6534725339011758

0.16022955651881965

0.5206693596399246

0.32777281162209315

.....



基本随机数函数

函数	描述
<code>seed(a=None)</code>	初始化给定的随机数种子，默认为当前系统时间 <code>>>>random.seed(10)</code> #产生种子10对应的序列
<code>random()</code>	生成一个[0.0, 1.0)之间的随机小数 <code>>>>random.random()</code> <code>0.5714025946899135</code>

基本随机数函数



```
>>> import random
```

```
>>> random.seed(10)
```

```
>>> random.random()
```

```
0.5714025946899135
```

```
>>> random.random()
```

```
0.4288890546751146
```

```
...
```

```
>>> import random
```

```
>>> random.seed(10)
```

```
>>> random.random()
```

```
0.5714025946899135
```

```
>>> random.seed(10)
```

```
>>> random.random()
```

```
0.5714025946899135
```

扩展随机数函数





扩展随机数函数

函数	描述
<code>randint(a, b)</code>	生成一个[a, b]之间的整数 <code>>>>random.randint(10, 100)</code> 64
<code>randrange(m, n[, k])</code>	生成一个[m, n)之间以k为步长的随机整数 <code>>>>random.randrange(10, 100, 10)</code> 80



扩展随机数函数

思考：如何从字符串中随机选取一个字符或指定长度的子串？

```
import random as rd
s="人生苦短，我学python"
a=rd.randint(0, 12)
s[a]
'我'
a=rd.randint(0, 12)
s[a]
'学'
```

```
b=rd.randint(0, 8)
s[b:b+4]
', 我学p'
b=rd.randint(0, 8)
s[b:b+4]
'ytho'
```



扩展随机数函数

函数	描述
getrandbits(k)	生成一个k比特长的随机整数 <pre>>>>random.getrandbits(16)</pre> <pre>37885</pre>
uniform(a, b)	生成一个[a, b]之间的随机小数 <pre>>>>random.uniform(10, 100)</pre> <pre>13.096321648808136</pre>



扩展随机数函数

函数	描述
choice(seq)	从序列seq中随机选择一个元素 <pre>>>>random.choice([1,2,3,4,5,6,7,8,9])</pre> 8
shuffle(seq)	将序列seq中元素随机排列，返回打乱后的序列 <pre>>>>s=[1,2,3,4,5,6,7,8,9];random.shuffle(s);print(s)</pre> [3, 5, 8, 9, 6, 1, 2, 7, 4]



随机数函数的使用

需要掌握的能力

- 能够利用随机数种子产生"确定"伪随机数
- 能够产生随机整数
- 能够对序列类型进行随机操作

随机数函数使用实例

编写程序，随机产生骰子的一面（数字1--6），给用户**三次**猜测机会，程序给出猜测提示（偏大或偏小）。如果某次猜测正确，则提示正确并结束循环；如果三次均猜错，则提示机会用完



```
import random
```

```
# 生成随机骰子点数（1-6）
```

```
dice_value = random.randint(1, 6)
```

```
# 初始化猜测次数和是否猜中的标志
```

```
guess_count = 0
```

```
# 主循环 - 最多3次猜测机会
```

```
while guess_count < 3:
```

```
    # 获取用户输入
```

```
    guess = int(input("请输入你的猜测: "))
```

```
    # 判断猜测结果
```

```
    if guess == dice_value:
```

```
        print("恭喜你! 猜对了!")
```

```
        guessed_correctly = True
```

```
        break
```

```
    elif guess < dice_value:
```

```
        print("猜小了, 再试试!")
```

```
    else:
```

```
        print("猜大了, 再试试!")
```

```
    guess_count += 1
```

```
else:
```

```
    print("你三次机会用完了")
```

```
print("游戏结束, 谢谢参与!")
```





单元小结: random库和随机数运用

- 使用random库产生随机数
- seed(), random(), randint(), getrandbits()
- uniform(), randrange(), choice(), shuffle()



本章小结



分支结构及异常处理

- 单分支 *if* 二分支 *if-else* 及紧凑形式
- 多分支 *if-elif-else* 及条件之间关系
- *not and or > >= == <= < !=*
- 异常处理 *try-except-else-finally*



循环结构

- *for...in* 遍历循环: 计数、字符串、列表、文件...
- *while* 无限循环
- *continue* 和 *break* 保留字: 退出当前循环层次
- 循环 *else* 的高级用法: 与 *break* 有关



条件计算

- 逻辑运算 *not and or*
- 比较运算 *> >= == <= < !=*



本章学习资源

配套微视频资源：第4章 程序的控制结构

单分支结构



二分支结构



多分支结构



程序的异常和处理



遍历循环



无限循环



循环控制保留字



循环的高级用法



条件判断及组合



身体质量指数BMI
问题分析



身体质量指数BMI
实例讲解



身体质量指数BMI
举一反三



圆周率的计算
问题分析



圆周率的计算
实例讲解



圆周率的计算
举一反三



random库
基本介绍



基本随机数函数



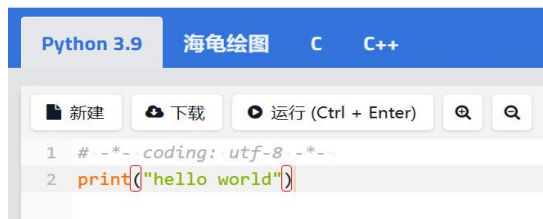
扩展随机数函数



在线实践平台：从 Python123 到 Learnverse

Python123
编程更简单

Python123



<https://python123.io>

智能升级

UPGRADE



Hi~ 欢迎来到

领航星 学习实践平台

为您提供大计算、大智能时代的全新学习体验

```
1 print("Hello World!")
2 print("Hello New World!")
3 print("Hello Python World!")
4 print("Hello New Python World!")
5
```

运行 复制代码



领航星 Learnverse

<https://www.learnverse.cn/>

在线优质课程：Python语言程序设计



国家精品在线开放课程

(请选择最新学期)



中国大学MOOC

下载手机APP